



tomato-haha

新随笔

管理

公告

昵称：tomato-haha
园龄：1年10个月
粉丝：13
关注：0
[+加关注](#)

< 2025年4月 >						
日	一	二	三	四	五	六
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

随笔分类

c/c++(18)
dds(3)
epoll(7)
linux-kernel(5)
physx(1)
python(12)
ubuntu(3)
UE4(49)
vim(1)
编码(3)
材质(1)
仿真(1)
内存对齐(1)
设计模式(2)
图形学(1)
虚函数(8)

随笔档案

2023年11月(1)
2023年10月(8)
2023年9月(18)
2023年8月(9)
2023年7月(1)
2023年6月(23)
2023年5月(56)

阅读排行榜

1. 分布式实时通信—DDS概述(12974)
2. linux swap 内存交换分区 详细介绍(2432)
3. windows10下安装、配置vim及YCM安装(1705)
4. C++11中的智能指针shared_ptr、weak_ptr源码解析(1556)

随笔 - 116 文章 - 0 评论 - 4 阅读 - 54794

分布式实时通信—DDS概述

<https://blog.yanjingang.com/?p=6716>

分布式实时通信—DDS概述

DDS最早应用在美国海军系统，目前已广泛应用于军工、机器人、自动驾驶等领域，国际主流军工厂商、ROS2操作系统等都将DDS作为其控制系统的核心中间件，为各类战舰、机器人等提供可靠、高实时的分布式通信功能。今天就带大家了解一下。

目录 隐藏

- 一、概述
- 1.协议组成

2.通信模型

3.DDS与RTPS关系
- 二、应用领域
- 1.国防军工

2.机器人

3.自动驾驶

4.物联网
- 三、Fast-DDS
- 1.核心能力

2.总体架构

2.1.Entity实体

2.2.Policy策略

2.3.Domain域

2.3.1.Partitions分区

2.4.Publisher发布者

2.5.Subscriber订阅者

2.6.Topic主题

3.编程和执行模型

3.1.并发和多线程

3.2.事件驱动

4.功能

4.1.自发现协议

4.2.安全

4.3.日志

5.编译和测试

5.1.源码编译

5.1.1.Linux源码编译

5.1.2.Mac源码编译

5.2.应用层示例

5.3.使用idl生成应用层代码

5.4.应用层代码研读
- 四、总结

DDS（Data Distribution Service），即数据分发服务，是OMG（Object Management Group）对象管理组织发布的分布式通信规范，采用订阅发布模型，以中间件的形式提供通信服务，并提供QoS（Quality of Service）策略，保障数据实时、高效、灵活的分发。

5. C++多态虚函数表详解(多重继承、多继承情况)(1544)

推荐排行榜

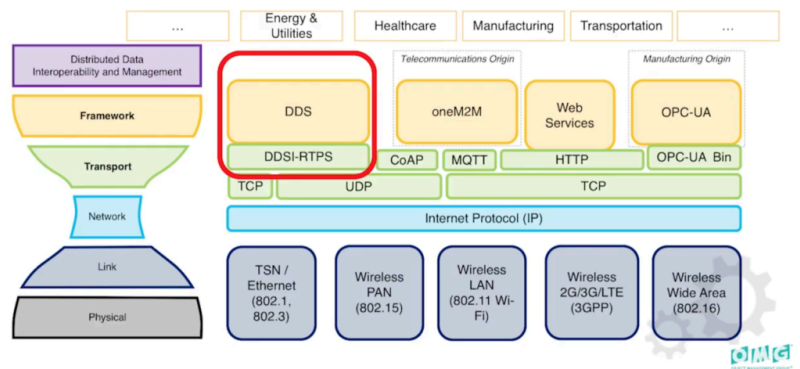
1. 分布式实时通信—DDS概述(2)
2. C++11中的智能指针shared_ptr、weak_ptr源码解析(1)
3. C++多态虚函数表详解(多重继承、多继承情况)(1)
4. 《asyncio 系列》3. 详解 Socket (阻塞、非阻塞), 以及和 asyncio 的搭配(1)
5. C++右值引用(1)

最新评论

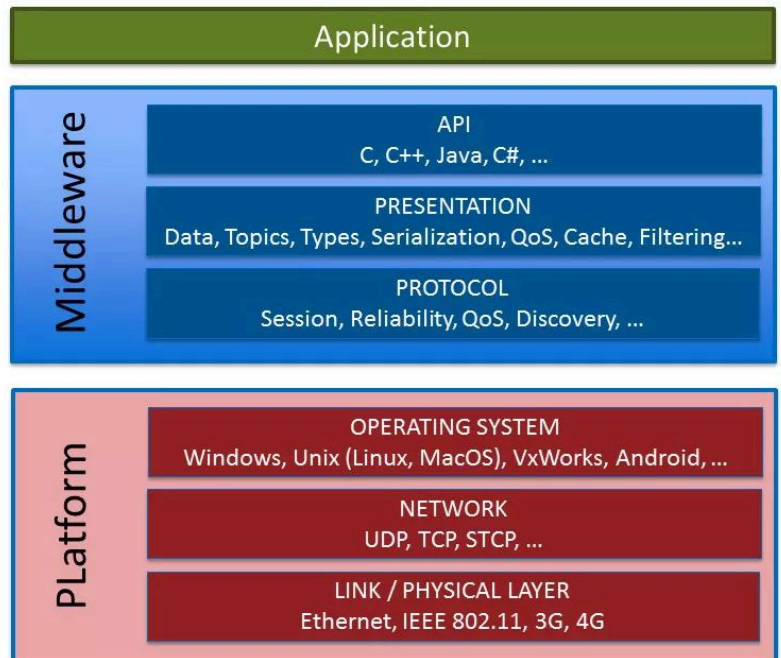
1. Re:windows10下安装、配置vim及YCM安装
BundleInstall 命令来安装, 这是不是旧版本使用的命令? 新版本安装插件的命令是 PluginInstall吗?
--codigger
2. Re:windows10下安装、配置vim及YCM安装
关于更改vundle安装路径的方法: 在set rtp后加call vundle#rc("your custom path")即可
--HASss
3. Re:ANSI是什么编码?
写的非常通透!
--LZH_03
4. Re:UE4命令行编译工程入门
请问打包这个IOS包, 执行这个脚本是在 mac 机器上执行的吧?
--我是冒险王

1.协议组成

DDS在网络栈中处于传输层的上面, 以TCP/UDP为基础:

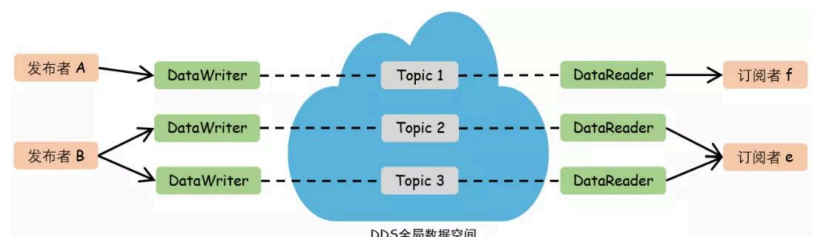


DDS的相关标准包括核心协议(DDSI-RTPS, DDS-XTypes, DDS-Security, Interface Definition Language (IDL)...), API(DDS C++ API, DDS Java API), 拓展协议(DDS-RPC, DDS-XML...)等13份协议集合。在分布式系统中, DDS位于操作系统和应用程序之间, 支持多种编程语言以及多种底层协议。

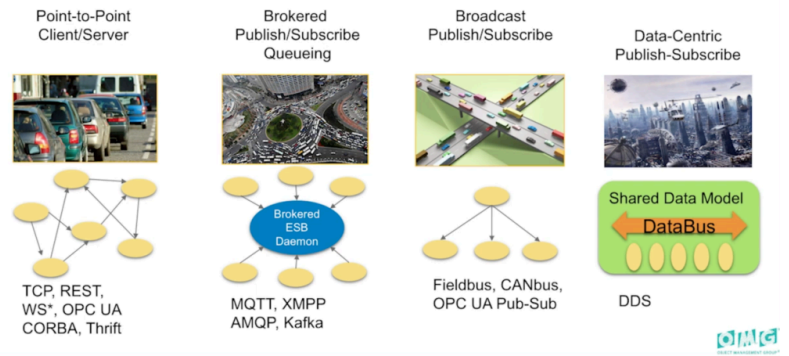


2.通信模型

DDS采用以数据为中心的发布-订阅模型DCPS (Data-Centric Publish-Subscribe):



与常见数据通信方式的对比:

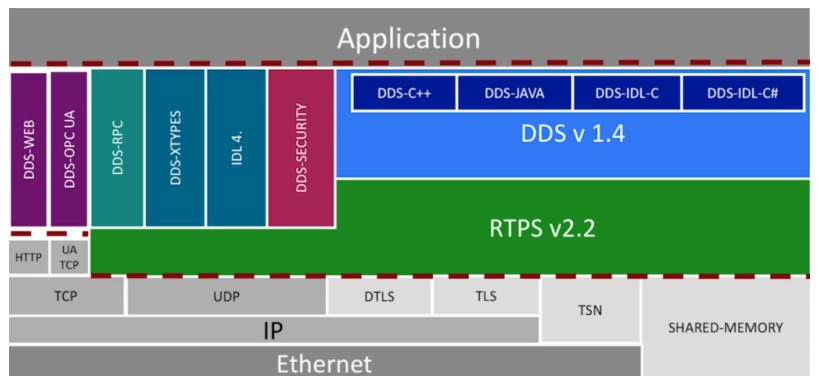


- Client-Server: 比较常见的TCP、WS、REST等均为这种请求响应模式；
- Broker: 消息队列，消息的生产和消费统一连接到broker，常见的MQTT、Kafka消息队列均属于这种模式；
- 广播: 每个人都可以在通道上广播和接受消息，通信双方没有直接连接，而是统一接到bus总线上收发，有点像买卖双方都要到跳蚤市场一样稍微有点乱，汽车上常见的CANbus即为这种模式；
- DDS: 与广播模型类似，但它可以在通信时只关心自己的消息，而不用看到那些不关心的嘈杂消息；

3.DDS与RTPS关系

在 DDS 规范中，有两个描述标准的基本文档：

- **DDS 规范**：DDS 规范描述了用于分布式应用程序通信和集成的以数据为中心的发布-订阅 (DCPS) 模型。该规范定义了应用程序接口 (API) 和通信语义 (行为和服务质量)，它们能够有效地将信息从信息生产者传递到匹配的消费者。DDS 规范的目的可以概括为“在正确的时间将正确的信息高效、稳健地传递到正确的地点”。
- **DDSI-RTPS**：RTPS (Real Time Publish Subscribe Protocol) 协议通过TCP/UDP等实现最大努力可靠的发布-订阅。该规范定义了 DDS 的互操作性协议，其目的和范围是确保基于不同供应商的 DDS 实现的应用程序可以互操作。

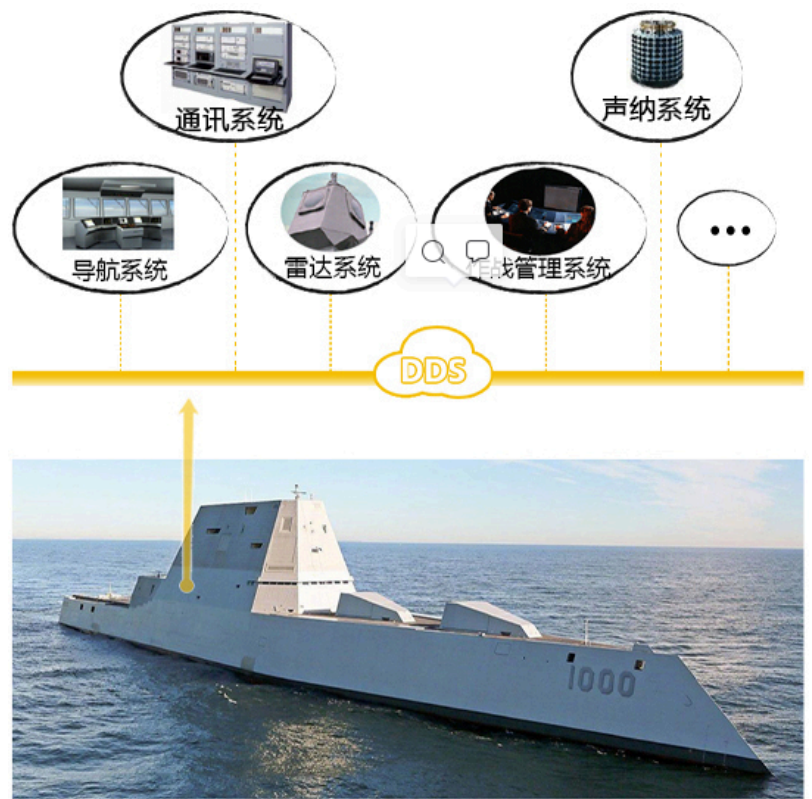


DDS本身是OMG维护的标准，具体有很多开源和商业产品实现了这个标准，比如FastRTPS、OpenDDS、OpenSplice DDS、Cyclone DDS等。

1.国防军工

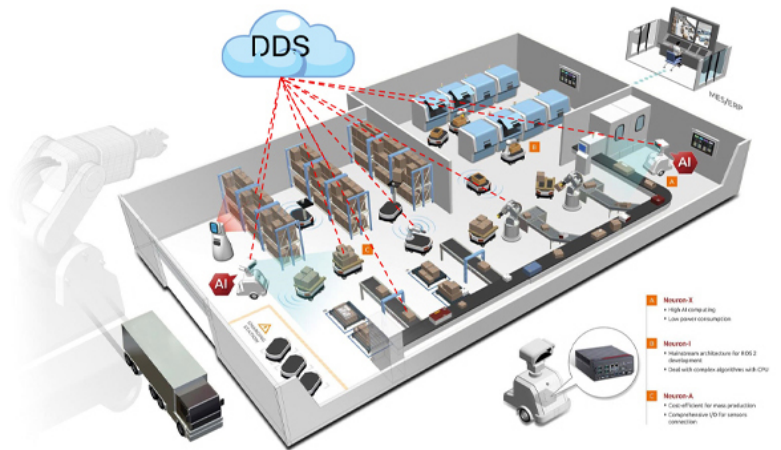
DDS最早应用在美国海军系统，用于解决军舰系统复杂网络环境中大量软件间的兼容性问题。目前DDS在欧美已广泛应用于国防军工领域，THALES泰雷兹、EADS欧洲宇航防务、Raytheon雷神、Lockheed Martin洛克希德·马丁等国际主流军工厂商均将DDS作为其各种作战指挥与控制系统的核心中间件，为F124/宙斯盾DDG1000护卫舰、MQ-9捕食者无人机、波音预警机、MK-41舰载导弹发射系统、

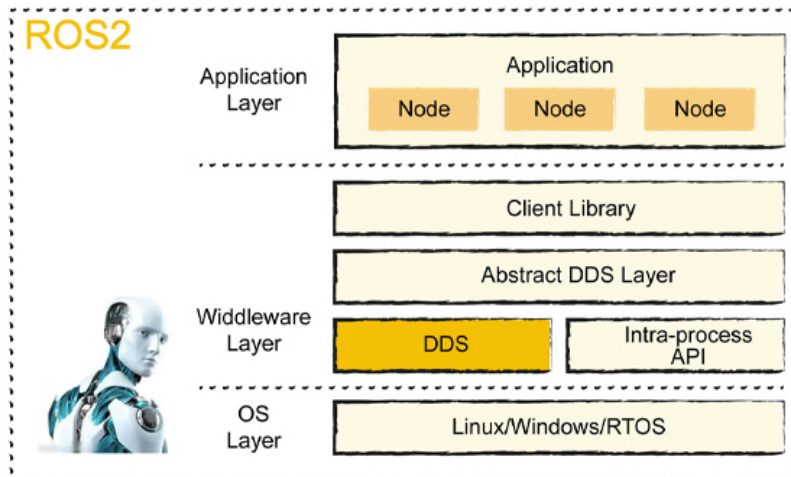
SSDS舰艇自卫系统、美国和欧洲宇航局发射控制系统等提供可靠、高实时的分布式通信功能。



2. 机器人

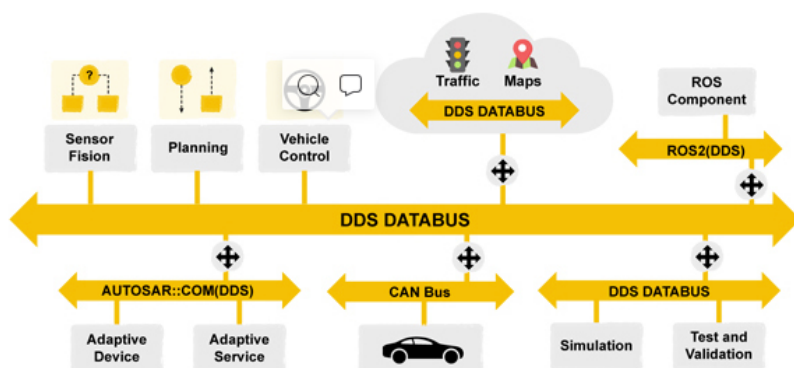
在机器人领域，传统单机作业模式已发展为群体分布式协同，对于生产作业等环节中的机器人集群，存在相互协作的可靠、灵活、实时、安全的连接需求，在ROS2中也将DDS确定为通信层标准，促进了机器人产业的发展。





3.自动驾驶

自动驾驶汽车是复杂的分布式系统，它结合了视觉、雷达、GPS、导航、规划和控制等组件，这些模块必须组成安全可靠的系统，实时分析复杂的环境并对其作出正确反应。多家自动驾驶平台均采用DDS集成多种复杂易购传感器收集的信息，以支撑其作出及时正确的控制决策。



4.物联网

物联网是一个物-物相连的互联网，需要持久、可靠、高性能网络架构支持。GE 多种医疗设备之间已采用DDS进行通信，西门子风电机的巨型涡轮机内部控制、外部通过无线/卫星与发电厂数据传输也已广泛应用。

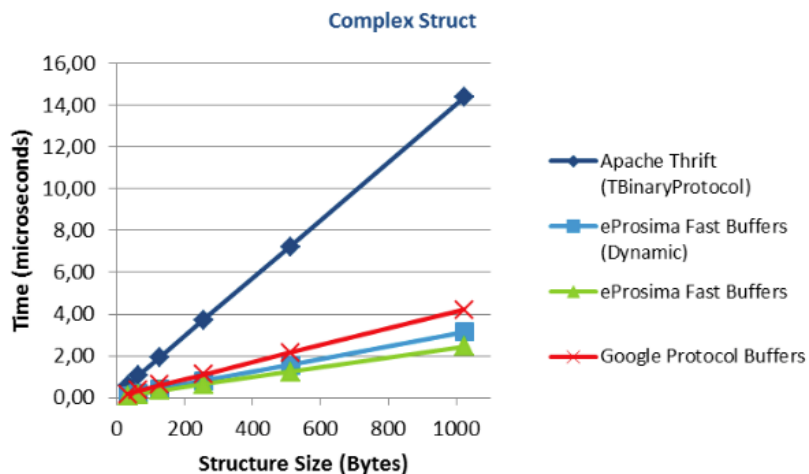
eprosima Fast DDS（以前称为 Fast-RTPS）是对OMG DDS标准的C++实现，是一个免费开源软件，遵循 Apache License 2.0，被ROS2设定为默认的消息中间件。

Fast-DDS 遵循 RTPS（实时发布订阅）协议，该协议通过不可靠传输（如 UDP）提供高效稳定的发布者-订阅者通信。RTPS 也是为数据分发服务 (DDS) 标准定义的互操作性协议。Fast-DDS公开了一个 API 以直接访问 RTPS 协议，使用户可以完全访问协议内部。

1.核心能力

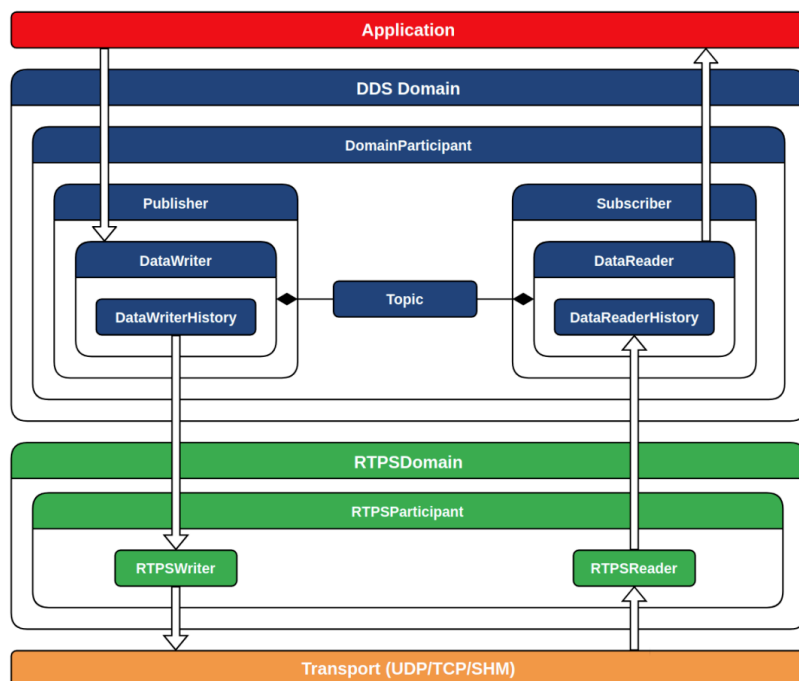
- **实时性**：提供可配置的实时、可靠的发布-订阅通信策略，支持同步和异步数据发布模式。
- **服务发现**：内置对发布者和订阅者的动态发现机制，即插即用连接，加入或离开网络无需配置。
- **流量控制**：支持可配置的吞吐量控制，可限制在特定条件下发送的数据量；

- **传输方式**：同时支持UDP/TCP/SHM(共享内存)传输；
- **安全性**：提供可配置的安全通信，包括远程参与者的身份验证、实体的访问控制和数据加密等；
- **分布式**：通过分布式网络可支持大量实体接入；
- **可移植性**：遵循DDS规范、IDL定义等，应用程序只需重新编译就可以在不同DDS实现之间切换；
- **高性能**：允许预分配资源，减少动态资源分配，避免无限制使用资源，最小化数据复制的需要；基于eProsima的Fast Buffers序列化，官网说其性能要高于Protocol Buffers和Thrift：



- **跨平台**：支持Linux、Mac OS、QNX、Windows、VxWorks、iOS、Android、Raspbian等平台。
- **免费和开源**：Fast DDS库、Fast RTPS库、Fast DDS-Gen、内部依赖项(如Fast CDR)和外部依赖项(如foonathan库)都是免费和开源的。

2. 总体架构



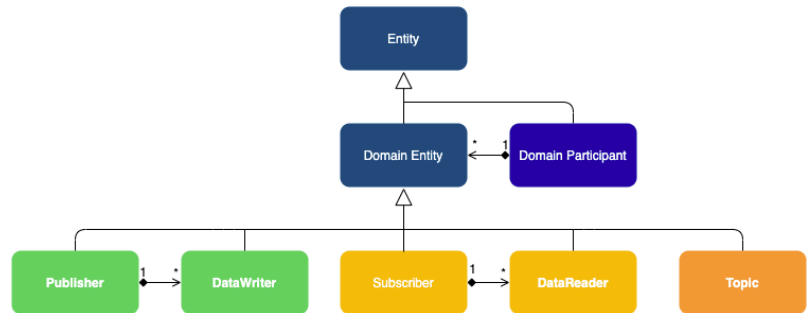
- **应用层**：使用Fast DDS API 在分布式系统中实现通信的用户应用程序。
- **DDS 层**：DDS 通信中间件的实现。它允许部署一个或多个 DDS 域，其中同一域内的域参与者通过在域主题下发布/订阅来交换消息。
- **RTPS 层**：实现实时发布-订阅 (RTPS) 协议以支持与 DDS 应用程序的互操作性。该层充当传输层的抽象层。
- **传输层**：快速 DDS可用于各种传输协议，例如不可靠传输协议 (UDP)、可靠传输协议 (TCP) 或共享内存传输协议 (SHM)。

2.1.Entity实体

Entity是所有 DDS 实体的抽象基类，DDS域下的所有对象都继承自Entity。包括以下类型：

- DomainParticipant：该实体是服务的入口点，充当发布者、订阅者和主题的工厂。
- Publisher：它充当可以创建任意数量的 DataWriters 的工厂。
- Subscriber：它充当可以创建任意数量的 DataReader 的工厂。
- Topic：该实体适合发布和订阅实体之间，并充当渠道。
- DataWriter：是负责数据分发的对象。
- DataReader：是用来访问接收到的数据的对象。

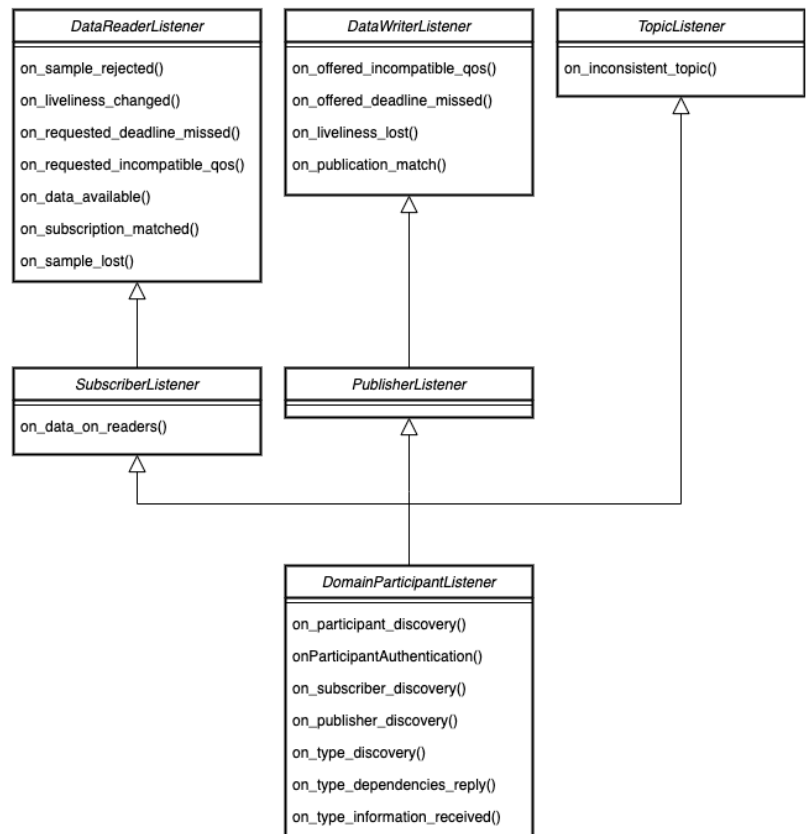
DDS 实体之间的层次结构：



所有实体共享的特征：

- 实体唯一ID：该 ID 在 DDS 实体及其对应的 RTPS 实体之间共享。ID 存储在 Entity 基类上声明的 Instance Handle 对象中，可以使用 getter 函数 `get_instance_handle()` 访问该对象。
- QoS策略：每个实体都可以使用策略配置，对于每一个实体类型都有对应的QoS类。在实体创建或使用中可以通过 `::set_qos()` 方法进行策略配置。
- Listener监听器：所有实体类型都定义了一个抽象侦听器接口，其中包含将实体状态变化触发传递给应用的回调函数。用户可以重写接口实现自己的侦听器回调。
- Status与StatusCondition：每个实体都与一组状态对象相关联，这些状态对象的值表示该实体的通信状态。这些状态值的更改会触发适当的侦听器回调的调用以异步通知应用程序。详见 [Status](#)、[Conditions](#) and [Wait-sets](#)。

监听器继承关系：



2.2.Policy策略

列几个常见的策略吧，相机的大家可以去看下[官方文档](#)：

- DeadlineQosPolicy：有效性策略，主要用于监控新数据的发布/接受频率，低于设定阈值时将发出报警。
- DestinationOrderQosPolicy：保序策略，用于同一个Topic的消息在reader时确保顺序。
- DurabilityQosPolicy：持久性策略，用于控制在没有reader的情况下，writer能否发送消息，以及后加入的reader能否读取加入前topic里的消息。
- EntityFactoryQosPolicy：实体工厂策略，用于disabled工具类的实体类。
- GroupDataQosPolicy：组策略，用于组合实现类似partition分布式的效果。
- HistoryQosPolicy：历史消息丢弃策略，控制是否只保留最新值以及保留的消息数量。
- LatencyBudgetQosPolicy：延迟策略，制定从写入数据到插入DataReader历史记录并通知到回调的最大可接受延迟。
- LivelinessQosPolicy：活跃策略，通过监控末次活跃后的等待时长来确定实体是否还活着。
- OwnershipQosPolicy：所有者策略，允许指定实例只能由一个DataWriter更新。
- PartitionQosPolicy：逻辑分区策略，DataReader只能接收Topic+指定Partition逻辑分区的信息。
- ReliabilityQosPolicy：可靠性策略，是否确保一定送达，并收到DataReader的确认。

2.3.Domain域

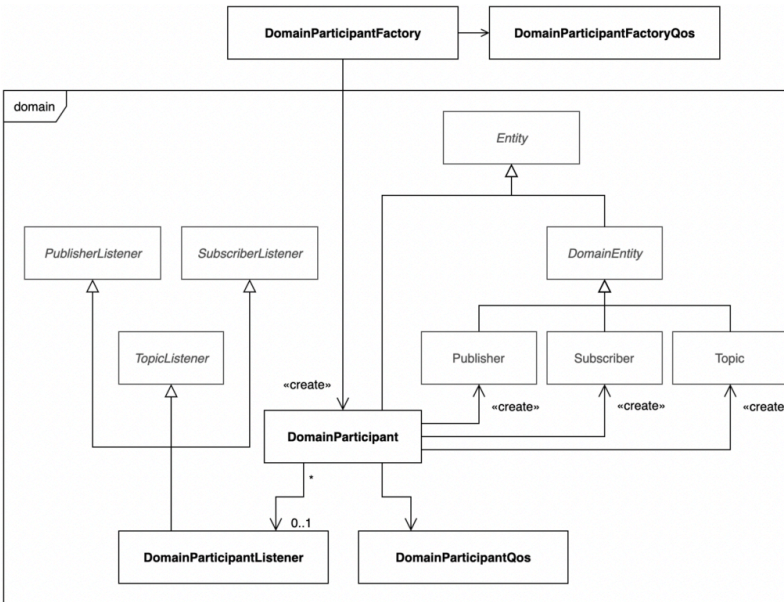
可以把域看作是一个虚拟网络，连接同域上运行的所有应用程序，并将它们与运行在不同域上的应用程序隔离开来。这样，多个独立的分布式应用程序可以在同一个物理网络中共存，而不会相互干扰，甚至不会相互感知。

每个域都有一个唯一的标识符，称为 domainId，它被实现为一个 `uint32` 值。同 domainId 的应用程序属于同一个域并且能够通信。

```
# 域参与者持有域domainId
DomainId_t did= 0;      # using DomainId_t = uint32_t;
```



```
participant = DomainParticipantFactory::get_instance()->create_participi
```



2.3.1.Partitions分区

分区在域中引入了另一个实体隔离级别。虽然 DomainParticipant 如果它们在同一域中将能够相互通信，但仍然可以隔离它们的 Publishers 和 Subscribers 并将它们分配给不同的 Partitions。

- 与域和主题不同，分区可以在端点的生命周期内以很少的成本动态更改。具体来说，不启动新线程，不分配新内存，不影响更改历史。
- 与 Domain 和 Topic 不同，一个端点可以同时属于多个 Partition。

可以在PublisherQos或SubscriberQos对象的PartitionQosPolicy 数据成员上配置分区。分区名为字符串，可以使用通配符，但需要注意*通配不匹配默认的无名partition，即同topic尽可能要么pub/sub终端都用partition，要么都不用，尽量不混着用。

```

PublisherQos pub_11_qos;
pub_11_qos.partition().push_back("Partition_1");
pub_11_qos.partition().push_back("Partition_2");

PublisherQos pub_12_qos;
pub_12_qos.partition().push_back("");

PublisherQos pub_21_qos;
//No partitions defined for pub_21

PublisherQos pub_22_qos;
pub_22_qos.partition().push_back("Partition*");

SubscriberQos subs_31_qos;
subs_31_qos.partition().push_back("Partition_1")

SubscriberQos subs_32_qos;
subs_32_qos.partition().push_back("Partition_2")

SubscriberQos subs_33_qos;
subs_33_qos.partition().push_back("Partition_3")

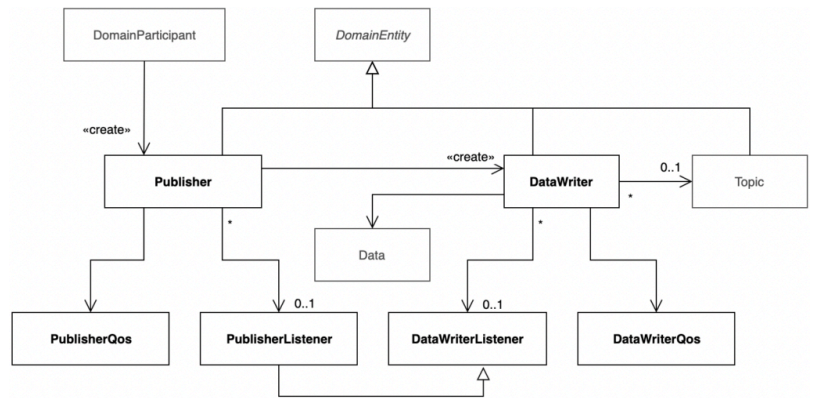
SubscriberQos subs_34_qos;
//No partitions defined for subs_34

```

	Participant_1		Participant_2		Participant_3			
	Pub_11	Pub_12	Pub_21	Pub_22	Subs_31	Subs_32	Subs_33	Subs_34
Partition_1	✓	✓	×	✓	✓	×	×	×
Partition_2	✓	✓	×	✓	×	✓	×	×
Partition_3	×	✓	×	✓	×	×	✓	×
{default}	×	×	×	×	×	×	×	✓

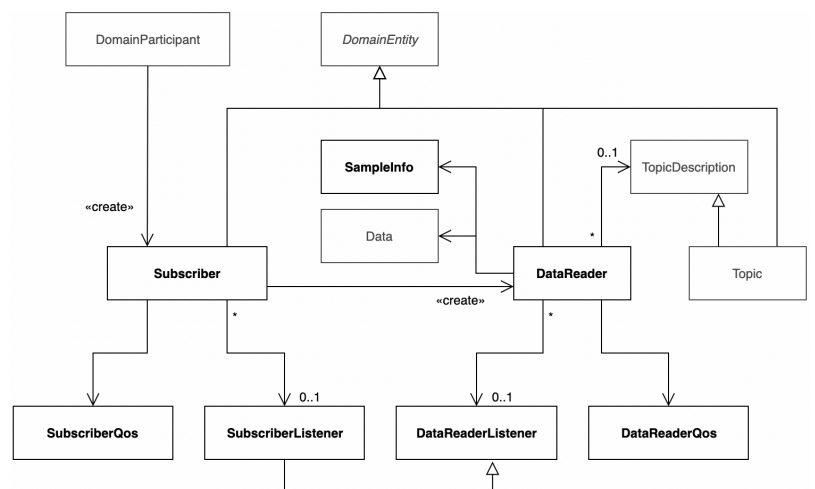
2.4.Publisher发布者

消息发布由Publisher和DataWriter共同完成。Publisher通过DataWriter向Topic发布消息，两者通过Listener监听进行异步回调处理。



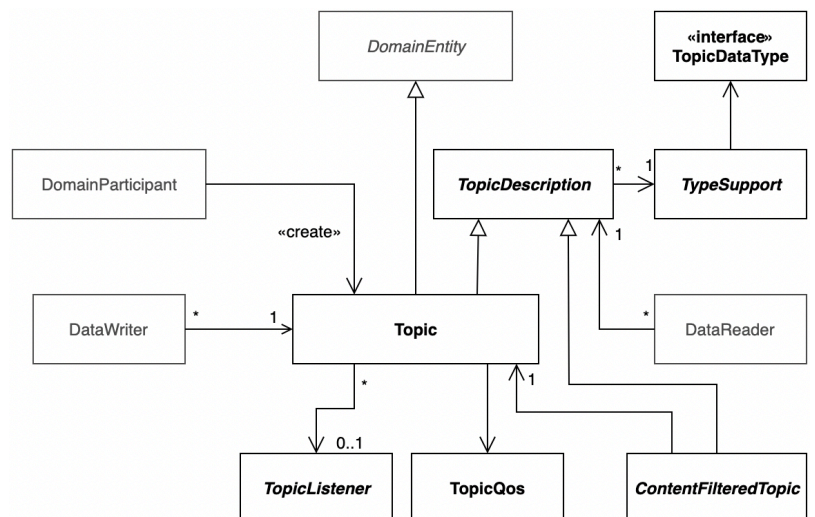
2.5.Subscriber订阅者

订阅由Subscriber和DataReader共同完成。Subscriber创建DataReader，DataReader绑定Topic，当接收到数据时，通过Listener通知应用进行回调处理。

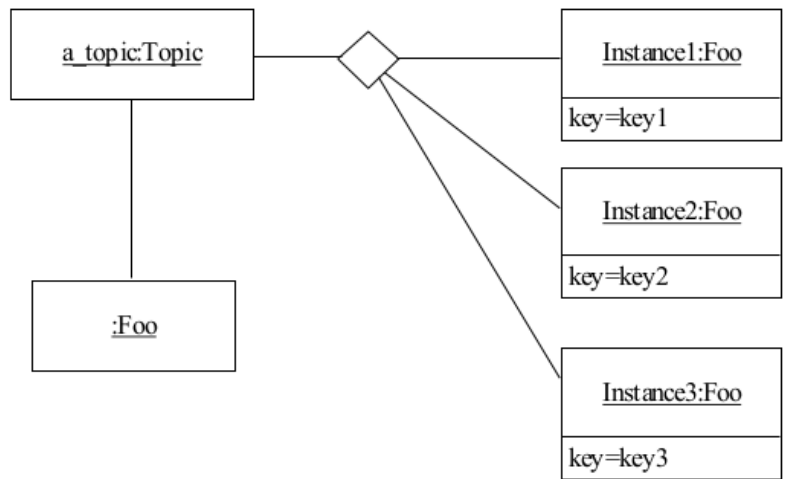


2.6.Topic主题

Topic用来连接同一类数据的发布方和订阅方。



单个Topic绑定到单个数据类型，所以每条数据可以理解为一个数据类型的实例：



Subscriber在订阅topic时，除了基本的订阅方式外，还可以选择提供数据筛选能力的contentFilteredTopic，它提供类似SQL的过滤参数配置：

```

// 域成员
DomainParticipant* participant =
    DomainParticipantFactory::get_instance()->create_participant(0,
if (nullptr == participant)
{
    // Error
    return;
}

// 话题
/* IDL
 *
 * struct HelloWorld
 * {
 *     long index;
 *     string message;
 * }
 */
Topic* topic =
    participant->create_topic("HelloWorldTopic", "HelloWorld", TOPIC_QOS_DEFAULT);
if (nullptr == topic)
{
    // Error
    return;
}

// 通过表达式筛选topic数据内容
std::string expression = "message like 'Hello*'";
std::vector<std::string> parameters;
ContentFilteredTopic* filter_topic =
    participant->create_contentfilteredtopic("HelloWorldFilteredTopic", "HelloWorld", TOPIC_QOS_DEFAULT);
if (nullptr == filter_topic)
{
    // Error
    return;
}

// 通过表达式+参数筛选topic数据内容
expression = "message like %0 or index > %1";
parameters.push_back("'*world*')");
parameters.push_back("20");
ContentFilteredTopic* filter_topic_with_parameters =
    participant->create_contentfilteredtopic("HelloWorldFilteredTopic", "HelloWorld", TOPIC_QOS_DEFAULT);
if (nullptr == filter_topic_with_parameters)
{
    // Error
    return;
}

// 订阅者
Subscriber* subscriber =
    participant->create_subscriber(SUBSCRIBER_QOS_DEFAULT);
  
```

```

if (nullptr == subscriber)
{
    // Error
    return;
}

// 带数据过滤能力的DataReader1
DataReader* reader_on_filter = subscriber->create_datareader(filter_top
if (nullptr == reader_on_filter)
{
    // Error
    return;
}

// 带数据过滤能力的DataReader2
DataReader* reader_on_filter_with_parameters =
    subscriber->create_datareader(filter_topic_with_parameters, DAT
if (nullptr == reader_on_filter_with_parameters)
{
    // Error
    return;
}

```

你也可以自定义过滤策略，或结合业务场景在DataWriter端进行过滤。

3.编程和执行模型

3.1.并发和多线程

Fast DDS实现了一个并发多线程系统。每个DomainParticipant生成一组线程来处理后台任务，例如日志记录、消息接收和异步通信。这不会影响使用该库的方式，即Fast DDS API是线程安全的，因此可以从不同的线程安全的调用同一DomainParticipant上的任何方法。但是，当外部函数访问由库内运行线程修改的资源时，必须考虑这种多线程实现。这方面的一个示例是应用Listener侦听器回调中的修改资源。

Fast DDS多线程调度工作方式：

- **主线程**：由应用程序管理。
- **事件线程**：每个 DomainParticipant 拥有其中之一。它处理周期性和触发的时间事件。
- **异步写入线程**：该线程管理所有 DomainParticipants 的异步写入。即使对于同步写，某些形式的通信也必须在后台启动。
- **接收线程**：域参与者为每个接收通道生成一个线程，其中通道的概念取决于传输层（例如 UDP 端口）。

3.2.事件驱动

FastDDS通过一个时间事件系统使来响应某些条件并安排定期执行。由于大多数与 DDS 和 RTPS 元数据有关，因此用户很少看到它们。但是，用户可以通过在他们的应用程序中继承 **TimedEvent** 类实现周期性事件。

4.功能

4.1.自发现协议

自发现协议定义了相同 Topic 下的 DataWriters 和 DataReader 相匹配的机制，以便它们可以共享数据。Fast DDS提供以下发现机制：

- **简单发现**：这是默认的发现机制，在 RTPS 标准中定义并提供与其他 DDS 实现的兼容性。在这里，DomainParticipants 是在早期单独发现的，以便随后匹配它们实现的 DataWriter 和 DataReader。
- **发现服务器**：这种发现机制使用集中式发现架构，其中服务器充当元流量发现的中心。

- **静态发现**：这实现了 DomainParticipant 彼此之间的发现，但如果远程 DomainParticipant 事先知道这些实体，则可以跳过每个 DomainParticipant (DataReader/DataWriter) 中包含的实体的发现。
- **手动发现**：此机制仅与 RTPS 层兼容。它允许用户使用其选择的任何外部元信息通道手动匹配和取消匹配 RTPSParticipants、RTPSWriters 和 RTPSReaders。

*注：详细解释和配置可以在[Discovery](#)查看。

4.2.安全

Fast DDS可通过在三个级别配置来提供安全通信：

- **远程域参与者的身份验证**：DDS:Auth:PKI-DH插件使用受信任的证书颁发机构 (CA) 和 ECDSA 数字签名算法提供身份验证，以执行相互身份验证。它还使用椭圆曲线 Diffie-Hellman (ECDH) 密钥协议建立共享秘密。
- **实体的访问控制**：DDS:Access:Permissions插件在DDS 域和主题级别为 DomainParticipants 提供访问控制。
- **数据加密**：DDS:Crypto:AES-GCM-GMAC插件使用伽罗瓦计数器模式 (AES-GCM) 中的高级加密标准 (AES) 提供经过身份验证的加密。

*注：详见 [Security](#) 。

4.3.日志

Fast DDS提供了一个可扩展的日志系统。Log类公开了三个宏定义以简化其使用logInfo、logWarning和logError。它使用正则表达式提供按类别过滤，以控制日志的详细程度。日志系统配置的详细信息详见[Logging](#)。

5.编译和测试

Fast-DDS 的源代码位于 Github 上：

```
mkdir ~/project/Fast-DDS/src && cd ~/project/Fast-DDS/src
git clone https://github.com/eProsima/Fast-DDS.git fastrtps
# 依赖
git clone https://github.com/eProsima/foonathan_memory_vendor.git
git clone https://github.com/eProsima/Fast-CDR.git fastcdr
git clone https://github.com/eProsima/Fast-DDS-Gen.git fastddsgen
git clone https://github.com/eProsima/IDL-Parser.git fastddsgen/thirdparty
git clone https://github.com/google/googletest
```

5.1.源码编译

5.1.1.Linux源码编译

```
# 1. 安装依赖
sudo apt install -y cmake g++ python3-pip wget git
# asio异步网络IO库、tinyxml解析器
sudo apt install -y libasio-dev libtinyxml2-dev
# openssl
sudo apt install -y libssl-dev
# Libp、softhsm安全库
sudo apt install -y libp11-dev libengine-pkcs11-openssl
sudo apt install -y softhsm2
sudo usermod -a -G softhsm $USER
# 检查libp能否找到softhsm
p11-kit list-modules
# 检查OpenSSL 是否能够访问 PKCS#11 引擎
openssl engine pkcs11 -t

# gtest
git clone https://github.com/google/googletest src/googletest-distribut
mkdir build && cd build
cmake ..
```

```

make

# colcon、vcstool编译工具
pip3 install -U colcon-common-extensions vcstool

# jdk、gradle 编译dds生成用
sudo apt install -y openjdk-8-jdk # gradle
# download binary from https://services.gradle.org/distributions/gradle
sudo mkdir /opt/gradle
sudo unzip -d /opt/gradle gradle-6.4-bin.zip
ls /opt/gradle/gradle-6.4
# path
export PATH=$PATH:/opt/gradle/gradle-6.4/bin
# run
gradle -v

# 2.vcs拉取相关源码 (foonathan_memory_vendor、Fast-CDR、Fast-DDS、IDL-Parser)
mkdir -p ~/project/Fast-DDS && cd ~/project/Fast-DDS
wget https://raw.githubusercontent.com/eProsima/Fast-DDS/master/fastrtps
mkdir src
vcs import src < fastrtps.repos
# 注: Fast-DDS代码库会被重命名为fastrtps, Fast-CDR重命名为fastcdr等。

# 3.编译
# 3.1.colcon编译
colcon build
# LIBRARY_PATH
source ~/project/Fast-DDS/install/setup.bash

# 3.2.cmake编译 (与colcon二选一即可)
# foonathan_memory_vendor
cd ~/project/Fast-DDS/src/foonathan_memory_vendor
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=~/project/Fast-DDS/install -DBUILD_SHARED_LIBS=ON
sudo cmake --build . --target install
# Fast-CDR
cd ~/project/Fast-DDS/src/fastcdr
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=~/project/Fast-DDS/install
sudo cmake --build . --target install
# Fast-DDS
cd ~/project/Fast-DDS/src/fastrtps
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=~/project/Fast-DDS/install -DCMAKE_BUILD_TYPE=Release
sudo cmake --build . --target install
# *注: 如想安装到操作系统, 修改所有编译选项里的以下参数即可
# -DCMAKE_INSTALL_PREFIX=/usr/local/ -DBUILD_SHARED_LIBS=ON

# LIBRARY_PATH
export LD_LIBRARY_PATH=/home/work/project/Fast-DDS/install/lib/
or
export LD_LIBRARY_PATH=/usr/local/lib/

# 4.dds python
sudo apt install -y swig libpython3-dev
# build
cd ~/project/Fast-DDS
vim fastrtps.repos
fastdds_python:
  type: git
  url: https://github.com/eProsima/Fast-DDS-python.git
  version: main
#wget https://raw.githubusercontent.com/eProsima/Fast-DDS-python/main/f
vcs import src < fastrtps.repos
colcon build
# path
source ~/project/Fast-DDS/install/setup.bash
export LD_LIBRARY_PATH=/home/work/project/Fast-DDS/install/lib/

# 5.dds gen
# build
cd ~/project/
git clone --recursive https://github.com/eProsima/Fast-DDS-Gen.git
cd Fast-DDS-Gen

```



```
./gradlew assemble
# path
export PATH=$PATH:~/project/Fast-DDS-Gen/scripts
```

5.1.2.Mac源码编译

```
# 1.工具&依赖
# homebrew
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install.sh)"
# cmake、asio等
brew install cmake boost python3 wget      # 如果brew安装cmake失败，可以到https://cmake.org/files/v3.22/cmake-3.22.0-macosx10.15.pkg
brew install asio tinyxml2 openssl@1.1      # 如果brew安装asio失败，可以到https://github.com/asio/asio/releases/download/v1.16.0/asio-1.16.0-macosx10.15.pkg
# colcon、vcstool
pip3 install -U colcon-common-extensions vcstool

# 2.vcs拉取相关源码（foonathan_memory_vendor、Fast-CDR、Fast-DDS、IDL-Parser）
mkdir ~/project/Fast-DDS && cd ~/project/Fast-DDS
wget https://raw.githubusercontent.com/eProsima/Fast-DDS/master/fastrtps
mkdir src
vcs import src < fastrtps.repos
# 注：Fast-DDS代码库会被重命名为fastrtps，Fast-CDR重命名为fastcdr等。

# 3.1.colcon编译
colcon build
# LIBRARY_PATH
source ~/project/Fast-DDS/install/setup.bash

# 3.2.cmake编译（与colcon二选一即可）
# foonathan_memory_vendor
cd ~/project/Fast-DDS/src/foonathan_memory_vendor
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=~/project/Fast-DDS/install -DBUILD_SHARED_LIBS=ON
sudo cmake --build . --target install
# Fast-CDR
cd ~/project/Fast-DDS/src/fastcdr
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=~/project/Fast-DDS/install -DBUILD_SHARED_LIBS=ON
sudo cmake --build . --target install
# Fast-DDS
cd ~/project/Fast-DDS/src/fastrtps
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=~/project/Fast-DDS/install -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=ON
sudo cmake --build . --target install
# *注：如想安装到操作系统，修改所有编译选项里的以下参数即可
# -DCMAKE_INSTALL_PREFIX=/usr/local/ -DBUILD_SHARED_LIBS=ON

# LIBRARY_PATH
export LD_LIBRARY_PATH=/home/work/project/Fast-DDS/install/lib/
or
export LD_LIBRARY_PATH=/usr/local/lib/

# 4.dds idl gen
# 安装编译依赖 jdk、gradle
#jdk可下载mac dmg安装https://www.oracle.com/java/technologies/downloads/#jdk17-mac
# 编译fastddsgen
cd ~/project/Fast-DDS/src/fastddsgen
./gradlew assemble
```

5.2.应用层示例

```
# 编译示例
cd ~/project/Fast-DDS/src/fastrtps/examples/cpp/dds/HelloWorldExample
mkdir build && cd build
cmake ../
make
或
cd ~/project/Fast-DDS/
vim src/fastrtps/CMakeLists.txt
option(COMPLETE_EXAMPLES "Build example" ON)
option(INSTALL_EXAMPLES "Install example" ON)
colcon build
```

运行示例

```
./DDSHelloWorldExample publisher
Starting
Publisher running 10 samples.
Publisher matched.
Message: HelloWorld with index: 1 SENT
Message: HelloWorld with index: 2 SENT
Message: HelloWorld with index: 3 SENT
Message: HelloWorld with index: 4 SENT
Message: HelloWorld with index: 5 SENT
Message: HelloWorld with index: 6 SENT
Message: HelloWorld with index: 7 SENT
Message: HelloWorld with index: 8 SENT
Message: HelloWorld with index: 9 SENT
Message: HelloWorld with index: 10 SENT
./DDSHelloWorldExample subscriber
Starting
Subscriber running. Please press enter to stop the Subscriber
Subscriber matched.
Message HelloWorld 1 RECEIVED # 如果subscriber在publisher后启动, 偶发第1条消息sub端没收到的情况。
Message HelloWorld 2 RECEIVED
Message HelloWorld 3 RECEIVED
Message HelloWorld 4 RECEIVED
Message HelloWorld 5 RECEIVED
Message HelloWorld 6 RECEIVED
Message HelloWorld 7 RECEIVED
Message HelloWorld 8 RECEIVED
Message HelloWorld 9 RECEIVED
Message HelloWorld 10 RECEIVED
Subscriber unmatched.
```

如果反复多次尝试先启动 `publisher` 再启动 `subscriber` 时, 会发现偶发第1条消息sub端没收到的情况。其原因为subscriber的matched回调可能晚于publisher的matched约70ms, 此时如果没有qos策略保障, pub时, sub端可能还没matched, 会导致sub端丢失数据。验证pub/sub matched时间差及解决丢数据问题的方法如下:

```
# 1. 在pub/sub的match和收发消息的地方增加微秒时间戳打印
# pub
uint64_t ts = std::chrono::duration_cast<std::chrono::microseconds>(std::chrono::system_clock::now().time_since_epoch());
std::cout << ts << " Publisher matched." << std::endl;

uint64_t ts = std::chrono::duration_cast<std::chrono::microseconds>(std::chrono::system_clock::now().time_since_epoch());
std::cout << ts << " Message: " << hello_.message() << " with index: " << hello_.index() << std::endl;

# sub
uint64_t ts = std::chrono::duration_cast<std::chrono::microseconds>(std::chrono::system_clock::now().time_since_epoch());
std::cout << ts << " Subscriber matched." << std::endl;

uint64_t ts = std::chrono::duration_cast<std::chrono::microseconds>(std::chrono::system_clock::now().time_since_epoch());
std::cout << ts << " Message " << hello_.message() << " " << hello_.index() << std::endl;

# 启动
./DDSHelloWorldExample publisher
1670310329490553 Starting
1670310329500484 Publisher running 10 samples.
1670310332695000 Publisher matched.
1670310332702463 Message: HelloWorld with index: 1 SENT # 消息1的pub时间戳
1670310332802577 Message: HelloWorld with index: 2 SENT
1670310332902696 Message: HelloWorld with index: 3 SENT
1670310333002813 Message: HelloWorld with index: 4 SENT
1670310333102933 Message: HelloWorld with index: 5 SENT
1670310333203057 Message: HelloWorld with index: 6 SENT
1670310333303187 Message: HelloWorld with index: 7 SENT
1670310333403318 Message: HelloWorld with index: 8 SENT
1670310333503451 Message: HelloWorld with index: 9 SENT
1670310333603579 Message: HelloWorld with index: 10 SENT

./DDSHelloWorldExample subscriber
1670310332685584 Starting
1670310332694407 Subscriber running. Please press enter to stop the Subscriber
1670310332765089 Subscriber matched.
1670310332845676 Message HelloWorld 2 RECEIVED # 消息1丢失
1670310332902758 Message HelloWorld 3 RECEIVED
```

```

1670310333002843 Message HelloWorld 4 RECEIVED
1670310333102963 Message HelloWorld 5 RECEIVED
1670310333203090 Message HelloWorld 6 RECEIVED
1670310333303222 Message HelloWorld 7 RECEIVED
1670310333403352 Message HelloWorld 8 RECEIVED
1670310333503519 Message HelloWorld 9 RECEIVED
1670310333603613 Message HelloWorld 10 RECEIVED
1670310333703766 Subscriber unmatched.

# 2.增加可靠性QoS
# pub qos
    qos.history().depth = 3;
    qos.reliability().kind = RELIABLE_RELIABILITY_QOS;
# sub qos
    qos.history().depth = 3;
    qos.reliability().kind = RELIABLE_RELIABILITY_QOS;
# 启动即可保障数据不丢失
./DDSHelloWorldExample publisher
1670311352137530 Starting
1670311352144375 Publisher running 10 samples.
1670311353318030 Publisher matched.
1670311353345727 Message: HelloWorld with index: 1 SENT # 消息1的pub时间!
1670311353446010 Message: HelloWorld with index: 2 SENT
1670311353546344 Message: HelloWorld with index: 3 SENT
1670311353646654 Message: HelloWorld with index: 4 SENT
1670311353746932 Message: HelloWorld with index: 5 SENT
1670311353847223 Message: HelloWorld with index: 6 SENT
1670311353947544 Message: HelloWorld with index: 7 SENT
1670311354047823 Message: HelloWorld with index: 8 SENT
1670311354148167 Message: HelloWorld with index: 9 SENT
1670311354248508 Message: HelloWorld with index: 10 SENT

./DDSHelloWorldExample subscriber
1670311353309845 Starting
1670311353317186 Subscriber running. Please press enter to stop the Sub
1670311353388798 Subscriber matched.
1670311353470344 Message HelloWorld 1 RECEIVED # 确保消息1送达
1670311353546459 Message HelloWorld 2 RECEIVED
1670311353646771 Message HelloWorld 3 RECEIVED
1670311353747064 Message HelloWorld 4 RECEIVED
1670311353847348 Message HelloWorld 5 RECEIVED
1670311353947673 Message HelloWorld 6 RECEIVED
1670311354047942 Message HelloWorld 7 RECEIVED
1670311354148288 Message HelloWorld 8 RECEIVED
1670311354248627 Message HelloWorld 9 RECEIVED
1670311354349079 Subscriber unmatched.

```

5.3.使用idl生成应用层代码

```

# 根据idl生成代码
# define idl
cd ~/project/Fast-DDS/src/fastrtps/examples/cpp/dds/
mkdir test && cd test
cp ../HelloWorldExample/HelloWorld.idl ./Test.idl
vim Test.idl
    struct Test # 数据类型
    {
        @key unsigned long index; # 索引
        string message; # 消息
    };

# path
source /home/work/project/Fast-DDS/install/setup.bash
export LD_LIBRARY_PATH=/home/work/project/Fast-DDS/install/lib/
export PATH=$PATH:/home/work/project/Fast-DDS/src/fastddsgen/scripts

# gen
# 生成idl定义类型的c++类代码
fastddsgen Test.idl -d . -replace
    Test.h/cxx # 包含idl定义属性以及序列化方法的类
    TestPubSubTypes.h/cxx # 集成自fastdds::dds::TopicDataType的topic
# 生成应用调用示例代码
fastddsgen Test.idl -example CMake -d .

```

```
TestPublisher.h/cxx      # 发布类封装, 包含init()、run()方法, 重写on_publication_matched、on_subscription_matched、on_data_available方法
TestSubscriber.h/cxx     # 订阅类封装, 包含init()、run()方法, 重写on_publication_matched、on_subscription_matched、on_data_available方法
TestPubSubMain.cxx       # main入口, 实例化TestPublisher/TestSubscriber

# 生成python示例
#fastddsgen Test.idl -python -example CMake -d .

# 编译运行代码
cmake ..
cmake --build .
./Test publisher
./Test subscriber
```

5.4.应用层代码研读

代码太长不在此贴了, 简单来说 `TestPublisher`、`TestSubscriber` 主要做了这么两件事:

- 定义participant成员、subscriber订阅/publisher发布、data reader读/writer写、topic、type (idl类型)、listener监听器, 并在init()时初始化 (关系链与fastdds架构图相同), 在run()时进行消息的pub/sub处理。
- 重写data reader/writer listener监听器的on_publication_matched、on_subscription_matched、on_data_available回调方法, 在发布或新消息到来时, 进行相应的处理。

传输的消息格式类似protobuf, 在.idl内定义后, 通过fastddsgen自动生成对应类, 在应用中使用即可。

DDS在服务质量 (QoS) 上提供非常多的保障途径, 这也是它适用于国防军事、工业控制这些高可靠性、可安全性应用领域的原因。但这些应用都工作在有线网络下, 在无线网络, 特别是资源受限的情况下应用还比较少见。作为一个跨域通信中间件协议, 其非常适合需要同时支持进程内、进程间、跨域通信 并对实时性和可靠性要求较高的场景。

好了, 今天先到这里, 接下来我们将继续深入的研究下FastDDS的底层实现机制。

参考:

[一文读懂“数据分发服务DDS” \(Data Distribution Service, RTPS, OMG\)](#)

[DDS与FastRTPS](#)

[eProsima/Fast-DDS](#)

[Fast RTPS Installation from Sources](#)

[mac下cmake安装、配置和使用](#)

[Fast RTPS与Cyclone DDS与OpenSplice DDS对比测试](#)

分类: [dds](#)

好文要顶

关注我

收藏该文

微信分享



tomato-haha
粉丝 - 13 关注 - 0

2

0

[+加关注](#)

[升级成为会员](#)

« 上一篇: [\[FASTDDS\]02-安装fast-gen](#)

» 下一篇: [深入理解ICMP协议](#)

posted @ 2023-09-12 16:17 tomato-haha 阅读(12974) 评论(0) 编辑 收藏 举报

登录后才能查看或发表评论, 立即 [登录](#) 或者 [逛逛](#) 博客园首页

【推荐】100%开源! 大型工业跨平台软件C++源码提供, 建模, 组态!

【推荐】还在用 ECharts 开发大屏? 试试这款永久免费的开源 BI 工具!

【推荐】国内首个AI IDE, 深度理解中文开发场景, 立即下载体验Trae

【推荐】编程新体验, 更懂你的AI, 立即体验豆包MarsCode编程助手

【推荐】轻量又高性能的 SSH 工具 IShell: AI 加持, 快人一步



相关博文:

- Apollo: DDS与Fast-RTPS
- Linux2.1.13网络源代码学习(<https://qiankunli.github.io/2022/07/0...>)
- FastDDS-1.开始
- fastdds学习之3——库概览
- FastDDS-0.简介

阅读排行:

- BotSharp + MCP 三步实现智能体开发
- BotSharp 5.0 MCP: 迈向更开放的AI Agent框架
- 动物智能之数据标注员——狗篇
- 5. RabbitMQ 消息队列中 Exchanges(交换机) 的详细说明
- 设计模式脉络